

SQL Retail Analytics Project

1. Create Tables

```
CREATE TABLE customers (  
  customer_id NUMBER,  
  customer_name VARCHAR2(50)  
);
```

```
CREATE TABLE stores (  
  store_id NUMBER,  
  store_name VARCHAR2(50)  
);
```

```
CREATE TABLE products (  
  product_id NUMBER,  
  category VARCHAR2(50)  
);
```

```
CREATE TABLE sales (  
  sale_id NUMBER,  
  customer_id NUMBER,  
  store_id NUMBER,  
  product_id NUMBER,  
  sale_date DATE,  
  amount NUMBER,  
  quantity NUMBER,  
  discount NUMBER  
);
```

2. Insert Data (Auto Generate 1000 Rows)

```
-- Customers  
INSERT INTO customers  
SELECT LEVEL, 'Cust_'||LEVEL FROM dual CONNECT BY LEVEL<=50;
```

```
-- Stores  
INSERT INTO stores VALUES (1,'CMR');  
INSERT INTO stores VALUES (2,'SR');
```

```
INSERT INTO stores VALUES (3,'Lucky');
```

```
-- Products
```

```
INSERT INTO products VALUES (1,'Electronics');
```

```
INSERT INTO products VALUES (2,'Fashion');
```

```
INSERT INTO products VALUES (3,'Grocery');
```

```
INSERT INTO products VALUES (4,'Home');
```

```
-- Sales (1000 rows)
```

```
INSERT INTO sales
```

```
SELECT
```

```
LEVEL,
```

```
MOD(LEVEL,50)+1,
```

```
MOD(LEVEL,3)+1,
```

```
MOD(LEVEL,4)+1,
```

```
DATE '2025-01-01' + MOD(LEVEL,90),
```

```
ROUND(DBMS_RANDOM.VALUE(100,10000)),
```

```
MOD(LEVEL,5)+1,
```

```
CASE WHEN MOD(LEVEL,4)=0 THEN NULL ELSE ROUND(DBMS_RANDOM.VALUE(10,500))
```

```
END
```

```
FROM dual
```

```
CONNECT BY LEVEL <= 1000;
```

1.Data Understanding

Manager Requirement:

Give me overall business performance summary for the last quarter.

Questions:

1.Find total number of transactions.

```
SELECT COUNT(*) AS total_transactions FROM sales;
```

Output:

	TOTAL_TRANSACTIONS
1	2000

2.Find total revenue generated.

```
SELECT SUM(amount) AS total_revenue FROM sales;
```

Output:

	TOTAL_REVENUE
1	10180798

3.Find total unique customers.

```
SELECT COUNT(DISTINCT customer_id) AS unique_cust FROM sales;
```

Output:

	UNIQUE_CUST
1	50

4.Find average order value.

```
SELECT AVG(amount) AS avg_order_value FROM sales;
```

	AVG_ORDER_VALUE
1	5090.399

5.Find total quantity sold.

```
SELECT SUM(quantity) AS total_quantity FROM sales;
```

Output:

	TOTAL_QUANTITY
1	6000

6.Find minimum and maximum sale amount.

```
SELECT MIN(amount) AS min_sale, MAX(amount) AS max_sale  
FROM sales;
```

Output:

	MIN_SALE	MAX_SALE
1	108	9999

7.Find average discount given.

```
SELECT AVG(NVL(discount, 0)) AS avg_dis FROM sales;
```

Output:

	AVG_DIS
1	194.8265

8.Find total sales per store.

```
SELECT store_id, SUM(amount) AS total_sales FROM sales GROUP BY store_id;
```

Output:

	STORE_ID	TOTAL_SALES
1	1	3320860
2	2	3451260
3	3	3408678

9.Find total sales per category

```
SELECT p.category, SUM(s.amount) AS total_sales
FROM sales s
JOIN products p
ON s.product_id = p.product_id
GROUP BY p.category;
```

Output:

	CATEGORY	TOTAL_SALES
1	Electronics	5243052
2	Fashion	4928540
3	Grocery	5124222
4	Home	5065782

10.Find number of transactions per day.

```
SELECT sale_date, COUNT(*) AS transactions
FROM sales
GROUP BY sale_date
ORDER BY sale_date;
```

Output:

	SALE_DATE	TRANSACTIONS
1	1/1/2025, 12:00:00	11
2	1/2/2025, 12:00:00	12
3	1/3/2025, 12:00:00	12
4	1/4/2025, 12:00:00	12

2. Data Cleaning

Manager Requirement:

Data looks inconsistent, fix NULL issues before analysis.

Questions:

11.Replace NULL discount with 0.

```
SELECT sale_id, NVL(discount, 0) AS dis
FROM sales;
```

Output:

	SALE_ID	DIS
1	408	0
2	409	361
3	410	125
4	411	327

12.Find number of NULL discounts.

```
SELECT COUNT(*) AS null_dis
FROM sales
WHERE discount IS NULL;
```

Output:

	NULL_DIS
1	250

13.Calculate net revenue (amount - discount).

```
SELECT sale_id, amount - NVL(discount, 0) AS net_rev
FROM sales;
```

Output:

	SALE_ID	NET_REV
1	408	5406
2	409	5902
3	410	951
4	411	-94

14.Replace NULL quantity with 1.

```
SELECT sale_id, NVL(quantity, 1) AS quan
FROM sales;
```

Output:

	SALE_ID	QUAN
1	408	4
2	409	5
3	410	1
4	411	2

15. Identify rows where amount is NULL.

```
SELECT * FROM sales WHERE amount IS NULL;
```

Output:

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
No items to display.							

16. Use COALESCE to handle multiple NULL columns.

```
SELECT sale_id, COALESCE(amount, 0) AS amo, COALESCE(quantity, 1) AS quan,  
COALESCE(discount, 0) AS dis FROM sales;
```

Output:

	SALE_ID	AMO	QUAN	DIS
1	408	5406	4	0
2	409	6263	5	361
3	410	1076	1	125
4	411	233	2	327

17. Create a cleaned column for revenue.

```
SELECT sale_id, amount - NVL(discount, 0) AS cleaned_rev FROM sales;
```

Output:

	SALE_ID	CLEANED_REV
1	408	5406
2	409	5902
3	410	951
4	411	-94

18. Check percentage of missing data.

```
SELECT sale_id, discount,  
(SELECT ((COUNT(*) - COUNT(discount)) * 100) / COUNT(*) FROM sales) AS  
missing_per  
FROM sales;
```

Output:

	SALE_ID	DISCOUNT	MISSING_PER
1	408	(null)	25
2	409	361	25
3	410	125	25
4	411	327	25

19.Flag rows with missing values.

```
SELECT sale_id,
CASE
    WHEN amount IS NULL OR quantity IS NULL OR discount IS NULL
    THEN 'Missing'
    ELSE 'Not Missing'
END AS status
FROM sales;
```

Output:

	SALE_ID	STATUS
1	408	Missing
2	409	Not Missing
3	410	Not Missing
4	411	Not Missing
5	412	Missing

20.Prepare dataset for analysis.

```
SELECT sale_id,
    COALESCE(amount, 0) AS amount,
    COALESCE(quantity, 1) AS quantity,
    COALESCE(discount, 0) AS discount,
    COALESCE(amount, 0) - COALESCE(discount, 0) AS net_rev
FROM sales;
```

Output:

	SALE_ID	AMOUNT	QUANTITY	DISCOUNT	NET_REV
1	408	5406	4	0	5406
2	409	6263	5	361	5902
3	410	1076	1	125	951
4	411	233	2	327	-94

3. Filtering (WHERE)

Manager Requirement:

I want specific insights based on conditions.

Questions:

21.Find sales in last 30 days.

```
SELECT * FROM sales
WHERE sale_date >= SYSDATE - 30;
```

Output:

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY
No items to display.						

22.Find sales above 5000.

```
SELECT * FROM sales WHERE amount > 5000;
```

Output:

	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	408	9	1	1 2/18/2025, 12:00:00	5406	4	(null)
2	409	10	2	2 2/19/2025, 12:00:00	6263	5	361
3	412	13	2	1 2/22/2025, 12:00:00	9496	3	(null)
4	414	15	1	3 2/24/2025, 12:00:00	8804	5	68

23.Find sales between 1000 and 5000.

```
SELECT sale_id, amount FROM sales WHERE amount BETWEEN 1000 AND 5000;
```

Output:

	SALE_ID	AMOUNT
1	410	1076
2	413	4954
3	416	1144
4	417	3918

24.Find sales from CMR store.

```
SELECT s.sale_id, s.amount, st.store_name
FROM sales s
JOIN stores st
ON s.store_id = st.store_id
WHERE st.store_name = 'CMR';
```

Output:

	SALE_ID	AMOUNT	STORE_NAME
1	408	5406	CMR
2	411	233	CMR
3	414	8804	CMR
4	417	3918	CMR

25.Find Electronics category sales.

```
SELECT s.sale_id, s.amount, p.category
FROM sales s
JOIN products p
ON s.product_id = p.product_id
WHERE p.category = 'Electronics';
```

Output:

	SALE_ID	AMOUNT	CATEGORY
1	408	5406	Electronics
2	412	9496	Electronics
3	416	1144	Electronics
4	420	8476	Electronics

26.Find sales excluding Grocery category.

```
SELECT s.sale_id, s.amount, p.category
FROM sales s JOIN products p ON s.product_id = p.product_id
WHERE p.category != 'Grocery';
```

Output:

	SALE_ID	AMOUNT	CATEGORY
1	408	5406	Electronics
2	409	6263	Fashion
3	411	233	Home
4	412	9496	Electronics

27.Find customers from specific store.

```
SELECT DISTINCT c.customer_id, c.customer_name
FROM sales s
JOIN customers c ON s.customer_id = c.customer_id
JOIN stores st ON s.store_id = st.store_id
WHERE st.store_name = 'CMR';
```

Output:

	CUSTOMER_ID	CUSTOMER_NAME
1	9	Cust_9
2	12	Cust_12
3	15	Cust_15
4	18	Cust_18

28.Find high discount transactions.

```
SELECT sale_id, amount, discount FROM sales
WHERE NVL(discount, 0) > 300;
```

Output:

	SALE_ID	AMOUNT	DISCOUNT
1	409	6263	361
2	411	233	327
3	413	4954	422
4	415	5971	344

29.Find low quantity sales.

```
SELECT sale_id, quantity FROM sales
WHERE quantity < 5;
```

Output:

	SALE_ID	QUANTITY
1	408	4
2	410	1
3	411	2
4	412	3

30.Find recent transactions.

```
SELECT sale_id, sale_date, amount FROM sales
WHERE sale_date >= (SELECT MAX(sale_date) - 7 FROM sales);
```

Output:

	SALE_ID	SALE_DATE	AMOUNT
1	442	3/24/2025, 12:00:00	1931
2	443	3/25/2025, 12:00:00	3864
3	444	3/26/2025, 12:00:00	7289
4	445	3/27/2025, 12:00:00	2647

4. Aggregation (GROUP BY)

Manager Requirement:

Break down performance across business segments.

Questions:

31.Revenue per store.

```
SELECT store_id, SUM(amount) AS revenue FROM sales
GROUP BY store_id;
```

Output:

	STORE_ID	REVENUE
1	1	1694870
2	2	1739098
3	3	1681030

32.Revenue per category.

```
SELECT p.category, SUM(s.amount) AS revenue
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.category;
```

Output:

	CATEGORY	REVENUE
1	Electronics	1250586
2	Fashion	1263408
3	Grocery	1261951
4	Home	1339053

33.Revenue per customer.

```
SELECT customer_id, SUM(amount) AS revenue FROM sales  
GROUP BY customer_id;
```

Output:

	CUSTOMER_ID	REVENUE
1	9	67810
2	10	113510
3	11	104530
4	12	97946

34.Monthly revenue.

```
SELECT TO_CHAR(sale_date, 'YYYY-MM') AS month, SUM(amount) AS revenue  
FROM sales  
WHERE sale_date BETWEEN DATE '2025-03-01' AND DATE '2025-03-31'  
GROUP BY TO_CHAR(sale_date, 'YYYY-MM');
```

Output:

	MONTH	REVENUE
1	2025-03	1811720

35.Daily revenue.

```
SELECT sale_date, SUM(amount) AS revenue FROM sales  
GROUP BY sale_date  
ORDER BY sale_date;
```

Output:

	SALE_DATE	REVENUE
1	1/1/2025, 12:00:00	63573
2	1/2/2025, 12:00:00	71098
3	1/3/2025, 12:00:00	66653
4	1/4/2025, 12:00:00	59902

36.Average sales per store.

```
SELECT store_id, AVG(amount) AS avg_sales FROM sales
GROUP BY store_id;
```

Output:

	STORE_ID	AVG_SALES
1	1	189.6996996996995
2	2	206.880239520958
3	3	148.138138138138

37.Total quantity per category.

```
SELECT p.category, SUM(s.quantity) AS total_quantity
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.category;
```

Output:

	CATEGORY	TOTAL_QUANTITY
1	Electronics	750
2	Fashion	750
3	Grocery	750
4	Home	750

38.Total discount per store.

```
SELECT store_id, SUM(NVL(discount, 0)) AS total_discount
FROM sales
GROUP BY store_id;
```

Output:

	STORE_ID	TOTAL_DISCOUNT
1	1	61535
2	2	66069
3	3	66593

39.Customer-wise total spending.

```
SELECT customer_id, SUM(amount) AS total_spen
FROM sales
GROUP BY customer_id;
```

Output:

	CUSTOMER_ID	TOTAL_SPEN
1	9	67810
2	10	113510
3	11	104530
4	12	97946

40.Store-wise transaction count.

```
SELECT store_id, COUNT(*) FROM sales GROUP BY store_id
```

Output:

	STORE_ID	COUNT(*)
1	1	666
2	2	668
3	3	666

5. HAVING

Manager Requirement:

Find top performing segments.

Questions:

41.Customers with revenue > 20000.

```
SELECT customer_id, SUM(amount) AS total FROM sales
GROUP BY customer_id
HAVING SUM(amount) > 20000;
```

Output:

	CUSTOMER_ID	TOTAL
1	9	181512
2	10	207612
3	11	195279
4	12	212332

42.Stores with revenue > 100000.

```
SELECT store_id, SUM(amount) AS rev FROM sales
GROUP BY store_id
HAVING SUM(amount) > 100000;
```

Output:

	STORE_ID	REV
1	1	3320860
2	2	3451260
3	3	3408678

43.Categories with avg sales > 3000.

```
SELECT product_id, AVG(amount) AS avg_sales FROM sales
GROUP BY product_id
HAVING AVG(amount) > 3000;
```

Output:

	PRODUCT_ID	AVG_SALES
1	1	5243.052
2	2	4928.54
3	3	5124.222
4	4	5065.782

44. Customers with more than 5 transactions.

```
SELECT customer_id, COUNT(*) FROM sales
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

Output:

	CUSTOMER_ID	COUNT(*)
1	9	40
2	10	40
3	11	40
4	12	40

45. Stores with high discount usage.

```
SELECT store_id, SUM(NVL(discount,0)) AS high_dis FROM sales
GROUP BY store_id
HAVING SUM(NVL(discount,0)) > 5000;
```

Output:

	STORE_ID	HIGH_DIS
1	1	128428
2	2	130341
3	3	130884

46. Categories with low performance.

```
SELECT product_id, SUM(amount) AS low_per FROM sales
GROUP BY product_id
HAVING SUM(amount) < 2000;
```

Output:

PRODUCT_ID	LOW_PER
------------	---------

No items to display.

47. Customers with high avg order value.

```
SELECT customer_id, AVG(amount) AS high_avg FROM sales
GROUP BY customer_id
HAVING AVG(amount) > 3000;
```

Output:

	CUSTOMER_ID	HIGH_AVG
1	9	4537.8
2	10	5190.3
3	11	4881.975
4	12	5308.3

48. Find top 5 customers using HAVING.

```
SELECT customer_id, SUM(amount) AS total
FROM sales
GROUP BY customer_id
ORDER BY total DESC
FETCH FIRST 5 ROWS ONLY;
```

Output:

	CUSTOMER_ID	TOTAL
1	25	247605
2	15	234619
3	21	229717
4	5	228274
5	26	225024

49. Stores with max transactions.

```
SELECT store_id, COUNT(*) cnt
FROM sales
GROUP BY store_id
HAVING COUNT(*) = (
```

```

SELECT MAX(cnt) FROM (
    SELECT COUNT(*) cnt FROM sales GROUP BY store_id
)
);

```

Output:

	STORE_ID	CNT
1	2	668

50.Categories with high growth.

```

SELECT product_id, SUM(amount)
FROM sales
GROUP BY product_id
HAVING SUM(amount) > 30000;

```

Output:

	PRODUCT_ID	TOTAL
1	1	2621526
2	2	2464270
3	3	2562111
4	4	2532891

6. CASE WHEN

Manager Requirement:

Segment customers for business decisions.

Questions:

51. Classify customers as High/Medium/Low spenders.

```
SELECT customer_id,  
CASE  
WHEN SUM(amount) >= 40000 THEN 'High'  
WHEN SUM(amount) >= 20000 THEN 'Medium'  
ELSE 'Low'  
END AS category  
FROM sales  
GROUP BY customer_id;
```

Output:

	CUSTOMER_ID	CATEGORY
1	9	High
2	10	High
3	11	High
4	12	High
5	13	High

52. Classify transactions as Big/Small.

```
SELECT sale_id,  
CASE WHEN amount > 5000 THEN 'Big' ELSE 'Small' END AS transac FROM sales;
```

Output:

	SALE_ID	TRANSAC
1	408	Big
2	409	Big
3	410	Small
4	411	Small

53. Create discount flag.

```
SELECT sale_id,  
CASE WHEN discount > 0 THEN 'Yes' ELSE 'No' END AS dis  
FROM sales;
```

Output:

	SALE_ID	DIS
1	408	No
2	409	Yes
3	410	Yes
4	411	Yes

54.Segment stores based on revenue.

```
SELECT store_id,
CASE WHEN SUM(amount) > 100000 THEN 'Top' ELSE 'Normal' END AS segment
FROM sales
GROUP BY store_id;
```

Output:

	STORE_ID	SEGMENT
1	1	Top
2	2	Top
3	3	Top

55.Categorize customers based on frequency.

```
SELECT customer_id,
CASE WHEN COUNT(*) > 10 THEN 'Frequent' ELSE 'Rare' END AS cust_freq
FROM sales
GROUP BY customer_id;
```

Output:

	CUSTOMER_ID	CUST_FREQ
1	9	Frequent
2	10	Frequent
3	11	Frequent
4	12	Frequent

56.Create sales buckets.

```
SELECT sale_id,amount,
CASE
WHEN amount < 1000 THEN 'Low'
WHEN amount < 5000 THEN 'Medium'
ELSE 'High'
END AS sales_buck
```

FROM sales;

Output:

	SALE_ID	AMOUNT	SALES_BUCK
1	408	5406	High
2	409	6263	High
3	410	1076	Medium
4	411	233	Low

57. Identify premium customers.

```
SELECT customer_id,  
CASE WHEN SUM(amount) > 25000 THEN 'Premium' END AS prem_cust  
FROM sales GROUP BY customer_id;
```

Output:

	CUSTOMER_ID	PREM_CUST
1	9	Premium
2	10	Premium
3	11	Premium
4	12	Premium

58. Mark high-value transactions.

```
SELECT sale_id,  
CASE WHEN amount > 7000 THEN 'High Value' END AS high_val  
FROM sales;
```

Output:

	SALE_ID	HIGH_VAL
1	408	(null)
2	409	(null)
3	410	(null)
4	411	(null)
5	412	High Value

59. Classify categories based on performance.

```
SELECT product_id,  
CASE WHEN SUM(amount) > 20000 THEN 'Good' ELSE 'Average' END AS cat_per  
FROM sales GROUP BY product_id;
```

Output:

	PRODUCT_ID	CAT_PER
1	1	Good
2	2	Good
3	3	Good
4	4	Good

60.Create custom labels for reporting.

```
SELECT sale_id,
CASE WHEN quantity>3 THEN 'Bulk' ELSE 'Normal' END AS cust_label
FROM sales;
```

Output:

	SALE_ID	CUST_LABEL
1	408	Bulk
2	409	Bulk
3	410	Normal
4	411	Normal

7. JOINS (Multiple Tables)

Manager Requirement:

Combine customer, product, and store data for insights.

Questions:

61.Join sales with customers table.

```
SELECT s.sale_id, s.amount, c.customer_name
FROM sales s
JOIN customers c ON s.customer_id = c.customer_id;
```

Output:

	SALE_ID	AMOUNT	CUSTOMER_NAME
1	408	5406	Cust_9
2	408	5406	Cust_9
3	409	6263	Cust_10
4	409	6263	Cust_10
5	410	1076	Cust_11

62.Join sales with stores table.

```
SELECT s.sale_id, s.amount, st.store_name
FROM sales s
JOIN stores st ON s.store_id = st.store_id;
```

Output:

	SALE_ID	AMOUNT	STORE_NAME
1	408	5406	CMR
2	408	5406	CMR
3	409	6263	SR
4	409	6263	SR
5	410	1076	Lucky

63.Join sales with products table.

```
SELECT s.sale_id, s.amount, p.category
FROM sales s
JOIN products p ON s.product_id = p.product_id;
```

Output:

	SALE_ID	AMOUNT	CATEGORY
1	408	5406	Electronics
2	408	5406	Electronics
3	409	6263	Fashion
4	409	6263	Fashion
5	410	1076	Grocery

64.Find customer name with store name and sales.

```
SELECT s.sale_id,c.customer_name,st.store_name,s.amount
FROM sales s
JOIN customers c ON s.customer_id = c.customer_id
JOIN stores st ON s.store_id = st.store_id;
```

Output:

	SALE_ID	CUSTOMER_NAME	STORE_NAME	AMOUNT
1	408	Cust_9	CMR	5406
2	408	Cust_9	CMR	5406
3	408	Cust_9	CMR	5406
4	408	Cust_9	CMR	5406
5	409	Cust_10	SR	6263

65.Find category-wise revenue using joins.

```
SELECT p.category, SUM(s.amount) AS REV
FROM sales s JOIN products p ON s.product_id=p.product_id
GROUP BY p.category;
```


Output:

	CATEGORY	REV
1	Electronics	5243052
2	Fashion	4928540
3	Grocery	5124222
4	Home	5065782

66.Find store-wise customer count.

```
SELECT st.store_name, COUNT(DISTINCT s.customer_id) AS cust_count
FROM sales s JOIN stores st ON s.store_id=st.store_id
GROUP BY st.store_name;
```

Output:

	STORE_NAME	CUST_COUNT
1	CMR	50
2	SR	50
3	Lucky	50

67.Find top product per store.

```
SELECT store_id, product_id, SUM(amount) AS total_sales
FROM sales
GROUP BY store_id, product_id
HAVING SUM(amount) = (SELECT MAX(SUM(amount)) FROM sales s2 WHERE s2.store_id =
sales.store_id GROUP BY product_id);
```

Output:

	STORE_ID	PRODUCT_ID	TOTAL_SALES
1	1	1	886756
2	3	3	885310
3	2	1	899923

68.Find customers who purchased from multiple stores.

```
SELECT c.customer_id, c.customer_name FROM sales s
JOIN customers c ON s.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name HAVING COUNT(DISTINCT s.store_id) > 1;
```

Output:

	CUSTOMER_ID	CUSTOMER_NAME
1	9	Cust_9
2	10	Cust_10
3	11	Cust_11
4	12	Cust_12
5	13	Cust_13

69.Find sales with product category.

```
SELECT s.sale_id, s.store_id, s.product_id, p.category, s.amount FROM sales s
JOIN products p ON s.product_id = p.product_id;
```

Output:

	SALE_ID	STORE_ID	PRODUCT_ID	CATEGORY	AMOUNT
1	408	1	1	Electronics	5406
2	408	1	1	Electronics	5406
3	409	2	2	Fashion	6263
4	409	2	2	Fashion	6263
5	410	3	3	Grocery	1076

70.Find revenue per customer per store.

```
SELECT c.customer_name, st.store_name, SUM(s.amount) AS revenue
FROM sales s
JOIN customers c ON s.customer_id = c.customer_id
JOIN stores st ON s.store_id = st.store_id
GROUP BY c.customer_name, st.store_name;
```

Output:

	CUSTOMER_NAME	STORE_NAME	REVENUE
1	Cust_9	CMR	214696
2	Cust_10	SR	274340
3	Cust_11	Lucky	190164
4	Cust_12	CMR	221976
5	Cust_13	SR	253032

71.Find customers with no transactions (LEFT JOIN).

```
SELECT c.customer_id, c.customer_name FROM customers c
LEFT JOIN sales s ON c.customer_id = s.customer_id
```

```
WHERE s.customer_id IS NULL;
```

Output:

CUSTOMER_ID	CUSTOMER_NAME
No items to display.	

72.Find missing product mappings.

```
SELECT * FROM sales s LEFT JOIN products p ON s.product_id=p.product_id
WHERE p.product_id IS NULL;
```

Output:

No items to display.

73.Find store performance using joins.

```
SELECT st.store_name,
       SUM(s.amount) AS total_rev,
       COUNT(s.sale_id) AS total_transac
FROM sales s
JOIN stores st ON s.store_id = st.store_id
GROUP BY st.store_name;
```

Output:

	STORE_NAME	TOTAL_REV	TOTAL_TRANSAC
1	CMR	6641720	1332
2	SR	6902520	1336
3	Lucky	6817356	1332

74.Find cross-category purchases.

```
SELECT c.customer_id, c.customer_name
FROM sales s
JOIN customers c ON s.customer_id = c.customer_id
JOIN products p ON s.product_id = p.product_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(DISTINCT p.category) > 1;
```

Output:

	CUSTOMER_ID	CUSTOMER_NAME
1	9	Cust_9
2	10	Cust_10
3	11	Cust_11
4	12	Cust_12

75.Find total revenue using joins.

```
SELECT SUM(s.amount) AS total_revenue
FROM sales s
```

```
JOIN customers c ON s.customer_id = c.customer_id
JOIN products p ON s.product_id = p.product_id
JOIN stores st ON s.store_id = st.store_id;
```

Output:

	TOTAL_REVENUE
1	81446384

8. Window Functions

Manager Requirement:

Provide advanced analytics without losing row-level data.

Questions:

76. Calculate total spend per customer using window function.

```
SELECT sale_id, customer_id, amount,
SUM(amount) OVER (PARTITION BY customer_id) AS total_spend
FROM sales;
```

Output:

	SALE_ID	CUSTOMER_ID	AMOUNT	TOTAL_SPEND
1	450	1	7840	198003
2	500	1	9137	198003
3	550	1	9494	198003
4	600	1	2102	198003
5	650	1	5154	198003

77. Rank customers by revenue.

```
SELECT customer_id, SUM(amount) AS TOTAL,
RANK() OVER(ORDER BY SUM(amount) DESC) AS rank
FROM sales GROUP BY customer_id;
```

Output:

	CUSTOMER_ID	TOTAL	RANK
1	25	247605	1
2	15	234619	2
3	21	229717	3
4	5	228274	4

78. Find top customer per store.

```
SELECT * FROM (
SELECT store_id, customer_id, SUM(amount) AS TOTAL,
RANK() OVER(PARTITION BY store_id ORDER BY SUM(amount) DESC) RANK
FROM sales GROUP BY store_id, customer_id) WHERE RANK=1;
```

Output:

	STORE_ID	CUSTOMER_ID	TOTAL	RANK
1	1	35	92953	1
2	2	29	93000	1
3	3	25	94247	1

79.Calculate running total of sales.

```
SELECT sale_id, sale_date, amount,  
SUM(amount) OVER (ORDER BY sale_date) AS running_total FROM sales;
```

Output:

	SALE_ID	SALE_DATE	AMOUNT	RUNNING_TOTAL
1	900	1/1/2025, 12:00:00	1593	110976
2	630	1/1/2025, 12:00:00	1189	110976
3	720	1/1/2025, 12:00:00	8617	110976
4	810	1/1/2025, 12:00:00	241	110976

80.Find previous sale using LAG.

```
SELECT sale_id, amount,  
LAG(amount) OVER(ORDER BY sale_id) AS prev_sale FROM sales;
```

Output:

	SALE_ID	AMOUNT	PREV_SALE
1	1	7679	(null)
2	1	6838	7679
3	2	6814	6838
4	2	5259	6814

81.Find next sale using LEAD.

```
SELECT sale_id, amount,  
LEAD(amount) OVER(ORDER BY sale_id) AS NEXT_SALE FROM sales;
```

Output:

	SALE_ID	AMOUNT	NEXT_SALE
1	1	7679	6838
2	1	6838	6814
3	2	6814	5259
4	2	5259	804

82.Calculate sales growth.

```
SELECT sale_id, amount,amount - LAG(amount) OVER(ORDER BY sale_id)  
as SALE_GROWTH FROM sales;
```

Output:

	SALE_ID	AMOUNT	SALE_GROWTH
1	1	7679	(null)
2	1	6838	-841
3	2	6814	-24
4	2	5259	-1555

83.Find difference between transactions.

```
SELECT sale_id,amount - LAG(amount) OVER(ORDER BY sale_id)AS differ FROM sales;
```

Output:

	SALE_ID	DIFFER
1	1	(null)
2	1	-841
3	2	-24
4	2	-1555

84.Calculate cumulative revenue.

```
SELECT sale_id, sale_date, amount,SUM(amount) OVER (ORDER BY sale_date ROWS
BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_revenue FROM sales;
```

Output:

	SALE_ID	SALE_DATE	AMOUNT	CUMULATIVE_REVE
1	900	1/1/2025, 12:00:00	1593	1593
2	630	1/1/2025, 12:00:00	1189	2782
3	720	1/1/2025, 12:00:00	8617	11399
4	810	1/1/2025, 12:00:00	241	11640

85.Find moving average of sales.

```
SELECT sale_id, amount,
AVG(amount) OVER (ORDER BY sale_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS
moving_avg
FROM sales;
```

Output:

	SALE_ID	AMOUNT	MOVING_AVG
1	1	7679	7679
2	1	6838	7258.5
3	2	6814	'110.333333333333
4	2	5259	303.666666666667

86.Rank stores by revenue.

```
SELECT store_id, SUM(amount) AS total_revenue,
RANK() OVER (ORDER BY SUM(amount) DESC) AS rank FROM sales
GROUP BY store_id;
```

Output:

	STORE_ID	TOTAL_REVENUE	RANK
1	2	3451260	1
2	3	3408678	2
3	1	3320860	3

87.Find second highest sale.

```
SELECT *
FROM (
    SELECT sale_id, amount,
    RANK() OVER (ORDER BY amount DESC) AS rnk FROM sales
)
WHERE rnk = 2;
```

Output:

	SALE_ID	AMOUNT	RNK
1	855	9990	2

88.Partition data by store and analyze.

```
SELECT sale_id, store_id, amount,SUM(amount) OVER (PARTITION BY store_id) AS
store_total FROM sales;
```

Output:

	SALE_ID	STORE_ID	AMOUNT	STORE_TOTAL
1	924	1	3022	3320860
2	927	1	4331	3320860
3	930	1	1470	3320860
4	933	1	9356	3320860

89.Find customer ranking per store.

```
SELECT store_id, customer_id, SUM(amount) AS total_spend,
RANK() OVER (PARTITION BY store_id ORDER BY SUM(amount) DESC) AS rnk FROM sales
GROUP BY store_id, customer_id;
```

Output:

	STORE_ID	CUSTOMER_ID	TOTAL_SPEND	RNK
1	1	35	92953	1
2	1	17	91552	2
3	1	43	89941	3
4	1	5	86612	4

90.Calculate frequency using window.

```
SELECT sale_id, customer_id,
COUNT(*) OVER (PARTITION BY customer_id) AS frequency FROM sales;
```

Output:

	SALE_ID	CUSTOMER_ID	FREQUENCY
1	450	1	40
2	500	1	40
3	550	1	40
4	600	1	40

9. NTILE / Segmentation

Manager Requirement:

Segment customers for marketing.

Questions:

91.Divide customers into 4 segments.

```
SELECT customer_id, total_spend,
NTILE(4) OVER (ORDER BY total_spend DESC) AS segment
FROM (
    SELECT customer_id, SUM(amount) AS total_spend
    FROM sales
    GROUP BY customer_id
);
```

Output:

	CUSTOMER_ID	TOTAL_SPEND	SEGMENT
1	25	247605	1
2	15	234619	1
3	21	229717	1
4	5	228274	1

92.Identify top 25% customers.

```
SELECT *
FROM (
    SELECT customer_id, SUM(amount) AS total_spend,
    NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS seg FROM sales
```



```
GROUP BY customer_id
) WHERE seg = 1;
```

Output:

	CUSTOMER_ID	TOTAL_SPEND	SEG
1	25	247605	1
2	15	234619	1
3	21	229717	1
4	5	228274	1

93. Identify bottom 25% customers.

```
SELECT * FROM (
SELECT customer_id, SUM(amount) AS total_spend,
NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS seg FROM sales
GROUP BY customer_id
)WHERE seg = 4;
```

Output:

	CUSTOMER_ID	TOTAL_SPEND	SEG
1	48	190545	4
2	4	189511	4
3	30	189382	4
4	33	186757	4

94. Segment stores into 3 groups.

```
SELECT store_id, total_revenue,
NTILE(3) OVER (ORDER BY total_revenue DESC) AS segment
FROM (
SELECT store_id, SUM(amount) AS total_revenue
FROM sales
GROUP BY store_id
);
```

Output:

	STORE_ID	TOTAL_REVENUE	SEGMENT
1	2	3451260	1
2	3	3408678	2
3	1	3320860	3

95. Create quartiles based on revenue.

```
SELECT customer_id, total_spend,
NTILE(4) OVER (ORDER BY total_spend DESC) AS quartile
FROM (
```

```

SELECT customer_id, SUM(amount) AS total_spend FROM sales
GROUP BY customer_id
);

```

Output:

	CUSTOMER_ID	TOTAL_SPEND	QUARTILE
1	25	247605	1
2	15	234619	1
3	21	229717	1
4	5	228274	1

96.Segment customers per store.

```

SELECT store_id, customer_id, total_spend,
NTILE(4) OVER (PARTITION BY store_id ORDER BY total_spend DESC) AS segment
FROM (
SELECT store_id, customer_id, SUM(amount) AS total_spend FROM sales
GROUP BY store_id, customer_id
);

```

Output:

	STORE_ID	CUSTOMER_ID	TOTAL_SPEND	SEGMENT
1	1	35	92953	1
2	1	17	91552	1
3	1	43	89941	1
4	1	5	86612	1

97.Identify premium segment.

```

SELECT * FROM (
SELECT customer_id, SUM(amount) AS total_spend,
NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS seg FROM sales
GROUP BY customer_id
)WHERE seg = 1;;

```

Output:

	CUSTOMER_ID	TOTAL_SPEND	SEG
1	25	247605	1
2	15	234619	1
3	21	229717	1
4	5	228274	1

98.Find mid-level customers.

```

SELECT *
FROM (
SELECT customer_id, SUM(amount) AS total_spend,

```

```

NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS seg
FROM sales
GROUP BY customer_id
)
WHERE seg IN (2,3);

```

Output:

	CUSTOMER_ID	TOTAL_SPEND	SEG
1	12	212332	2
2	39	211929	2
3	17	211830	2
4	20	211795	2

99.Analyze bucket distribution.

```

SELECT seg, COUNT(*) AS customer_count
FROM (
SELECT customer_id,
NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS seg
FROM sales
GROUP BY customer_id
)
GROUP BY seg
ORDER BY seg;

```

Output:

	SEG	CUSTOMER_COUNT
1	1	13
2	2	13
3	3	12
4	4	12

100.Create marketing segments.

```

SELECT customer_id, total_spend,

```

```

CASE
    WHEN seg = 1 THEN 'Premium'
    WHEN seg = 2 THEN 'High Value'
    WHEN seg = 3 THEN 'Medium Value'
    ELSE 'Low Value'
    END AS segment_label
FROM (
    SELECT customer_id, SUM(amount) AS total_spend,
    NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS seg FROM sales
    GROUP BY customer_id
);

```

Output:

	CUSTOMER_ID	TOTAL_SPEND	SEGMENT_LABEL
1	25	247605	Premium
2	15	234619	Premium
3	21	229717	Premium
4	5	228274	Premium

10. FIRST_VALUE / LAST_VALUE

Manager Requirement:

Compare performance with best and worst.

Questions:

101.Find highest sale.

```

SELECT DISTINCT
FIRST_VALUE(amount) OVER (ORDER BY amount DESC) AS highest_sale FROM sales;

```

Output:

	HIGHEST_SALE
1	9999

102.Find lowest sale.

```

SELECT DISTINCT
FIRST_VALUE(amount) OVER (ORDER BY amount ASC) AS lowest_sale FROM sales;

```

Output:

	LOWEST_SALE
1	108

103.Compare each sale with highest.

```

SELECT sale_id, amount,
FIRST_VALUE(amount) OVER (ORDER BY amount DESC) AS highest_sale,
amount - FIRST_VALUE(amount) OVER (ORDER BY amount DESC)AS differ FROM sales;

```

Output:

	SALE_ID	AMOUNT	HIGHEST_SALE	DIFFER
1	408	5406	9999	-4593
2	409	6263	9999	-3736
3	410	1076	9999	-8923
4	411	233	9999	-9766

104.Compare each sale with lowest.

```
SELECT sale_id, amount,  
FIRST_VALUE(amount) OVER (ORDER BY amount ASC) AS lowest_sale,  
amount - FIRST_VALUE(amount) OVER (ORDER BY amount ASC) AS differ FROM sales;
```

Output:

	SALE_ID	AMOUNT	LOWEST_SALE	DIFFER
1	408	5406	108	5298
2	409	6263	108	6155
3	410	1076	108	968
4	411	233	108	125

105.Find highest sale per store.

```
SELECT store_id, sale_id, amount,  
FIRST_VALUE(amount) OVER (PARTITION BY store_id ORDER BY amount DESC)  
As high_sale FROM sales;
```

Output:

	STORE_ID	SALE_ID	AMOUNT	HIGH_SALE
1	1	924	3022	9999
2	1	927	4331	9999
3	1	930	1470	9999
4	1	933	9356	9999

106.Find lowest sale per store.

```
SELECT store_id, sale_id, amount,  
FIRST VALUE(amount) OVER (PARTITION BY store id ORDER BY amount ASC)  
AS lowest_sale FROM sales;
```

Output:

	STORE_ID	SALE_ID	AMOUNT	LOWEST_SALE
1	1	924	3022	108
2	1	927	4331	108
3	1	930	1470	108
4	1	933	9356	108

107.Compare category performance.

```
SELECT p.category,
SUM(s.amount) AS total_sales,
FIRST_VALUE(SUM(s.amount)) OVER (ORDER BY SUM(s.amount) DESC) AS best_cat_sale,
SUM(s.amount) - FIRST_VALUE(SUM(s.amount)) OVER (ORDER BY SUM(s.amount) DESC)
AS best
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.category;
```

Output:

	CATEGORY	TOTAL_SALES	BEST_CAT_SALE	BEST
1	Electronics	5243052	5243052	0
2	Fashion	4928540	5243052	-314512
3	Grocery	5124222	5243052	-118830
4	Home	5065782	5243052	-177270

108.Find best customer per store.

```
SELECT store_id, customer_id, total_spend FROM (
SELECT store_id, customer_id, SUM(amount) AS total_spend,
FIRST_VALUE(customer_id) OVER (PARTITION BY store_id ORDER BY SUM(amount) DESC)
AS best_customer FROM sales
GROUP BY store_id, customer_id
)WHERE customer_id = best_customer;
```

Output:

	STORE_ID	CUSTOMER_ID	TOTAL_SPEND
1	1	35	92953
2	2	29	93000
3	3	25	94247

109.Find worst performing category.

```
SELECT DISTINCT p.category,
FIRST_VALUE(SUM(s.amount)) OVER (ORDER BY SUM(s.amount) ASC) AS low_sales
FROM sales s
```

```
JOIN products p ON s.product_id = p.product_id
GROUP BY p.category;
```

Output:

	CATEGORY	LOW_SALES
1	Electronics	4928540
2	Fashion	4928540
3	Grocery	4928540
4	Home	4928540

110. Analyze gap between best and worst.

```
SELECT
    MAX(total_sales) AS best_sales,
    MIN(total_sales) AS worst_sales,
    MAX(total_sales) - MIN(total_sales) AS gap
FROM (
    SELECT p.category, SUM(s.amount) AS total_sales
    FROM sales s
    JOIN products p ON s.product_id = p.product_id
    GROUP BY p.category
);
```

Output:

	BEST_SALES	WORST_SALES	GAP
1	5243052	4928540	314512

11. Date Analysis

Manager Requirement:

Analyze time-based trends.

Questions:

111.Find daily sales.

```
SELECT sale_date, SUM(amount) AS daily_sales FROM sales  
GROUP BY sale_date ORDER BY sale_date;
```

Output:

	SALE_DATE	DAILY_SALES
1	1/1/2025, 12:00:00	110976
2	1/2/2025, 12:00:00	122024
3	1/3/2025, 12:00:00	135281
4	1/4/2025, 12:00:00	107127

112.Find monthly sales.

```
SELECT SUM(amount) AS monthly_sales FROM sales  
WHERE sale_date >= DATE '2025-01-01'AND sale_date < DATE '2025-02-01';
```

Output:

	MONTHLY_SALES
1	3571363

113.Find weekend sales.

```
SELECT sale_date, SUM(amount) AS weekend_sales FROM sales  
WHERE TO_CHAR(sale_date, 'DY') IN ('SAT', 'SUN') GROUP BY sale_date;
```

Output:

	SALE_DATE	WEEKEND_SALES
1	2/22/2025, 12:00:00	118781
2	2/23/2025, 12:00:00	105098
3	3/1/2025, 12:00:00	106073
4	3/2/2025, 12:00:00	114292

114.Find first sale date.

```
SELECT MIN(sale_date) AS first_sale_date FROM sales;
```

Output:

	FIRST_SALE_DATE
1	1/1/2025, 12:00:00

115.Find last sale date

```
SELECT MAX(sale_date) AS last_sale_date FROM sales;
```

Output:

	LAST_SALE_DATE
1	3/31/2025, 12:00:00

116.Find sales in specific month.


```
SELECT * FROM sales
WHERE TO_CHAR(sale_date, 'YYYY-MM') = '2025-01';
```

Output:

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	450	1	1	3	1/1/2025, 12:00:00	7840	1	112
2	451	2	2	4	1/2/2025, 12:00:00	9330	2	326
3	452	3	3	1	1/3/2025, 12:00:00	7314	3	(null)
4	453	4	1	2	1/4/2025, 12:00:00	6477	4	462
5	454	5	2	3	1/5/2025, 12:00:00	2521	5	204

117.Find sales growth month-wise.

```
SELECT month, total_sales,
total_sales - LAG(total_sales) OVER (ORDER BY month) AS growth
FROM (
    SELECT TO_CHAR(sale_date, 'YYYY-MM') AS month,
    SUM(amount) AS total_sales
    FROM sales
    GROUP BY TO_CHAR(sale_date, 'YYYY-MM')
);
```

Output:

	MONTH	TOTAL_SALES	GROWTH
1	2025-01	3571363	(null)
2	2025-02	3063031	-508332
3	2025-03	3546404	483373

118.Find inactive customers (30 days).

```
SELECT customer_id FROM sales
GROUP BY customer_id
HAVING MAX(sale_date) < SYSDATE - 30;
```

Output:

	CUSTOMER_ID
1	9
2	10
3	11
4	12
5	13

119.Find repeat customers.

```
SELECT customer_id, COUNT(*) AS transac_count FROM sales
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Output:

	CUSTOMER_ID	TRANSAC_COUNT
1	9	40
2	10	40
3	11	40
4	12	40
5	13	40

120.Find sales trend.

```
SELECT sale_date,
SUM(amount) AS daily_sales,
SUM(SUM(amount)) OVER (ORDER BY sale_date) AS cumul_sales
FROM sales
GROUP BY sale_date
ORDER BY sale_date;
```

Output:

	SALE_DATE	DAILY_SALES	CUM_SALES
1	1/1/2025, 12:00:00	110976	110976
2	1/2/2025, 12:00:00	122024	233000
3	1/3/2025, 12:00:00	135281	368281
4	1/4/2025, 12:00:00	107127	475408
5	1/5/2025, 12:00:00	140940	616348